Sprint #1: Inventory Creation Modal

🎯 **Problem Statement**

Customers can't perform quick, contextual tasks without abandoning their current screen because our platform lacks native modal dialog support. This gap forces users into full-page navigation, increasing cognitive load, risking data inconsistencies, slowing down critical workflows, and ultimately undermining productivity and product adoption.

Executive Summary

To support critical 2025 Big Bet user stories, we must ship a reusable modal/dialog in our no-code platform. The **Screen-as-Modal** approach delivers the fastest path to an MVP by leveraging existing navigation and action infrastructure, while laying a foundation for future enhancements.

Problem Space

Background & Rational

Our blue-sky and 2025-scope prototypes revealed that a modal/dialog component is not "nice-to-have" but essential for delivering core UX flows. Without native modals, users must navigate away or resort to fragile workarounds, undermining both usability and lovability of our product offerings.

- **Team Prioritization:**
  - While Customer Management generated the most modal use-cases, followed by Service Delivery; Sales Management's first three core features—multi-warehouse management, available inventory visibility, and Katana integration—are blocked without modal support.
  - Delivering this component now unlocks these high-impact workflows and empowers all teams in subsequent phases.

Figma

Restricted content

You'll need to request access or try a different account to view this preview.

Try another account

Modal Use-Cases

Goals

1. **Enable Overlay Dialogs:** Provide a reusable modal that can be invoked from any screen, allowing users to perform isolated tasks without leaving their current context.
2. **Promote Reuse:** Promote reuse by allowing any standalone screen to double as a modal.
3. **Unlock Core Workflows & Adoption:** Lay a foundation that unlocks these high-impact workflows and empowers all teams to adopt modals seamlessly in future phases.
4. **Maintain Low Dependencies:** Leverage existing navigation and action infrastructure to deliver an MVP rapidly, without requiring changes to the screen designer or App Builder widgets.

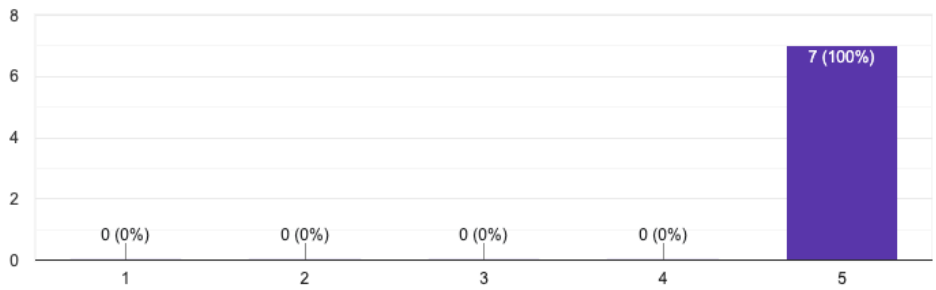
Time-to-Value

Consumer App Teams

To understand the immediate impact of the modal capability, we surveyed seven product professionals (product managers and designers)—and found unanimous enthusiasm (7/7 rated “5 – Very likely” to adopt if available today) and surfaced high-value use cases for immediate rollout.

Top Use Cases

- **Quick Add Flows**: Replace full-screen navigation with modals for quickly adding contacts, activities, or other entities while in the middle of other workflows (e.g., creating an activity and needing a new contact, or adding an activity from a contact list).
- **Creating New Records**: Utilize modals for adding new records from dropdown menus (e.g., new customer on an invoice screen) to prevent disrupting the user's current flow. This includes customer, lead, and vendor creation.
- **Editing Experiences**: Implement modals for editing emails and SMS messages from a "Send" button, offering a more streamlined experience.
- **Workflow Enhancements**: Convert current screen redirects to modals for a better and more streamlined experience, such as within a new visit experience from an estimate/work order, adding visits to the calendar or when converting a work order to an invoice.
- **Advanced Support Tools**: Update the "Advanced support tools" experience to use a modal instead of navigating to a different screen.



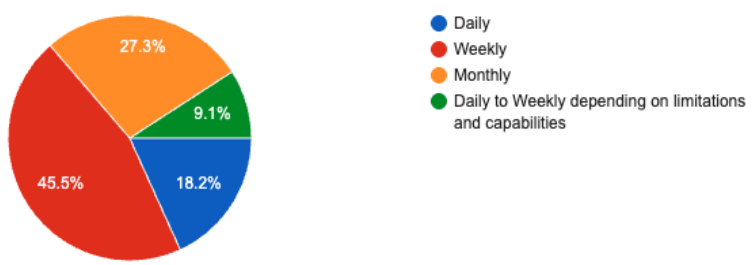
On a scale of 1–5, how likely would you be to implement the modal capability in your stock app flows if it were available today?

Additionally, here's a compiled a list of workflows where our existing "show message" feature hit its limits—forcing complex workarounds. These examples further underscore the need for modals to handle richer interactions without resorting to hacks.

PS Team

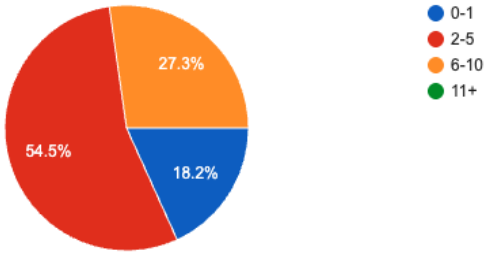
To understand the immediate impact of modal screens for Professional Services customizations, we surveyed eleven PS team members. The results demonstrate **immediate velocity & impact**:

1. **Adoption Cadence: 73% (8/11)** would use modals weekly or more, signalling strong anticipated adoption and regular use.
   a. 3 respondents (27%) expect to configure modals daily.
   b. 5 respondents (45%) expect to configure modals weekly.
   c. 3 respondents (27%) expect to configure modals monthly.



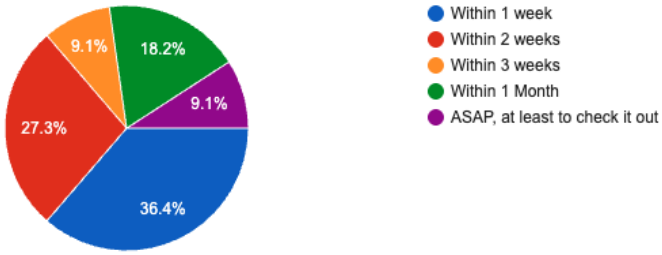
If the modal capability were available today, how frequently do you expect to configure modal screens in your customizations?

2. **Volume: 82% (9/11)** anticipate building at least 2 modals per week, indicating the feature will be actively used in customization workflows.
   a. 3 respondents (27%) expect to build 6–10 modals per week.
   b. 6 respondents (55%) expect 2–5 modals per week.
   c. 2 respondents (18%) expect 0–1 modals per week.



On average, how many modal screens would you expect to configure per week in your customization workflows, if it were available today?

3. **Time to Adoption: 73% (8/11)** would adopt within 2 weeks, demonstrating demand and a readiness to leverage modals quickly.
   a. 5 respondents (45%) would start using it within one week of release.
   b. 3 respondents (27%) would start within 2 weeks.
   c. 1 respondent (9%) would start within 3 week.
   d. 2 respondents (18%) would start within one month.



Once the modal feature is enabled for you, when would you realistically start using it?

Top Use Cases

1. **Transaction and Record Confirmations**

- Confirm critical actions such as signed estimates, waivers, reorders, batch activity updates, or merging records without leaving the current screen.
- Reduces errors and improves user confidence by keeping them oriented within their primary workflow.

2. **Editing or Updating Related Records In-Context**

- Enable quick edits to related records like customer information, adding multiple signatures, or attaching documents directly from a transaction screen.
- Eliminates the need to navigate away and return, which often disrupts a user's task and introduces errors.

3. **Gathering or Displaying Supplemental Information**

- Display or collect details such as checklists, embedded PDFs, serial numbers, catalog copies, or multiple-choice prompts in a pop-up.
- Provides a cleaner, focused user experience without overloading the main screen or forcing additional navigation.

4. **UI/UX Improvement**

- Optimizing screen real estate and decluttering the user interface by converting large sections into accessible pop-up screens, especially for rarely used features or information.

5. **Debugging & Troubleshooting**

- Employing modals for debugging and troubleshooting purposes.

The strong frequency, quick onboarding, and clear, high-impact scenarios demonstrate that PS teams will adopt modals—and that these interactions will drive real efficiency gains, reduce workarounds, and enhance end-user satisfaction across our customizations.

Solution Space

Approach: Screen-as-Modal (Decoupled Action-Based)

Treat a full screen as a modal overlay via a new “Open Screen in Pop Up” action in the Action Editor—mirroring existing “Show Message” functionality.

Advantages:

- **Rapid Delivery**: Leverages the existing no-code navigation Action Editor; helps unblock current blockers in days, not weeks.
- **High Reusability & Simplicity**: Any screen can be invoked anywhere—no section duplication.
- **Minimal Dependencies**: Decoupled from App Builder timeline; works in both legacy screen designer and app builder and keeps the Big Bet moving forward.

Treating a full screen as a modal reuses existing navigation logic, sidesteps WYSIWYG gaps, and naturally promotes reuse—making it the quickest way to unblock both feature development and the broader Big Bet initiatives.

Success Metrics

Category	Metric	Target	Rationale
Adoption DEFERRED	X% of eligible screens tagged as modal	≥X%	Tracks how quickly users (PS/customers) convert existing or

	within within 3 months of general availability		new screens into modals rather than re-building separate flows.
Adoption	# of screens with “Open Screen in Pop Up” action configured within 3 months of general availability	≥ 50	Reflects users’ willingness to rely on the modal feature rather than full-screen navigations.
Usage	Average # of unique users opening a modal per day within 3 months of general availability	≥ 50	Measures how many individual users (customers)—not just power users—are engaging with modals, which indicates broader adoption.
Business Impact	X% reduction in time to complete core workflows using modal versus previous full-screen flow	≥ X%	Directly reflects efficiency gains and validates the modal’s impact.

Functional Requirements

Below are the functional requirements (in-scope) that describe what the modal must do from a user’s perspective.

1. **Modal Configuration**

ID	User Story	Acceptance Criteria
FR-100 DEFERRED	As a user, I want to mark any existing/new screen as a “Modal” in the action editor, so that I can reuse that screen as an overlay without rebuilding it.	AC 1. **Toggle Availability** • **Given** I am on a screen’s Action Editor • **When** I view the screen’s configuration

options

- **Then** I see a “Mark as Modal” toggle that I can enable or disable.

2. **Visibility**

- **Given** I enabled “Mark as Modal” and saved changes
- **When** I reload the screen’s settings or revisit the Action Editor
- **Then** the “Mark as Modal” toggle remains enabled.

3. **Modal-Only Listing Downstream**

- **Given** I open any “Show Modal” action dropdown (after tagging screens)
- **When** I look at the list of screens
- **Then** only screens previously marked as “Modal” appear for selection.

4. **Multi-Functional**

- **Given** a screen is marked as “Modal”
- **When** a user navigates directly to that screen
- **Then** it renders as a standalone full-screen page (not as an overlay) and behaves

		identically to any non-modal screen.
FR-101	As a user, I want a “Open Screen in Pop Up” action in the Action Editor, so that I can invoke a modal overlay from any button, link, or action from the parent screen.	AC 1. **Screen Selection for Modal** DEFERRED • **Given** I select “Show Modal” as an action type • **When** I open the dropdown to choose a target screen • **Then** only screens marked as “Modal” (from User Story 1) are listed. 1. **Action Availability** • **Given** I am editing a button (or other clickable element) of a parent screen • **When** I open the Action Editor for that component and click “Add New Action” • **Then** “Open Screen in Pop Up” appears in the list of available actions, allowing you to choose which screen to overlay as a modal via a dropdown menu 2. **Runtime Behaviour**

		• **Given** I configure “Open Screen in Pop Up” and save • **When** I return to the parent screen at runtime and click the configured element • **Then** the specified modal screen overlays the parent as a centred dialog.
FR-102	As a user, I want a “Close Pop Up” action in the Action Editor, so that I can close a modal overlay from the close button within the modal.	AC 1. **Action Availability** • **Given** I am editing the a button • **When** I open the Action Editor for that component and click “Add New Action” • **Then** “Close Pop Up” appears in the list of available actions. 2. **Runtime Behaviour** • **Given** the modal is open and I click the element with the “Close Pop Up” action • **When** the action fires • **Then** the modal closes immediately, focus returns to a logical element on the parent screen, and any configured “On Pop Up Close” hooks execute.

3. **Show Message**

- **Given** no modal is currently open
- **When** the “Close Pop Up” action is triggered
- **Then** a show message appears saying “No pop up is open,” and no modal is closed.

4. **No Side Effects**

- **Given** no modal is open
- **When** the “Close Pop Up” action is triggered
- **Then** the UI state remains unchanged and no errors are thrown.

5. **Consistent Response on Repeated Triggers**

- **Given** no modal is open
- **When** the user triggers “Close Pop Up” multiple times in succession
- **Then** the same message appears each time and the system behaviour remains stable.

FR-103 DEFERRED	As a user, I want the modal feature gated by a feature flag with targeted enablement, so that only Method Support users and partner accounts can configure and use modal actions during initial rollout.	AC 1. Create a new feature flag a. Name: Open Screen in Pop Up b. Should be ON for the Method Support user for all accounts i. Existing ones ii. New signups moving forward c. Should be ON for the whole Account for all partner accounts i. Existing ones ii. New signups moving forward d. Should be OFF for all other accounts 2. When the feature flag above is set to ON, the “Open Screen in Pop Up” and “Close Pop Up” action can be selected and added to an action set

2. **Runtime Behaviour**

ID	User Story	Acceptance Criteria
FR-200	As a user, I want to see a screen appear as a centred overlay when “Open Screen in Pop Up” is triggered, so that I can interact with it, without leaving the parent screen.	AC 1. **Overlay Appearance** • **Given** a parent screen with a configured

		“Open Screen in Pop Up” action • **When** I click the element that triggers “Open Screen in Pop Up” • **Then** the screen renders as a centred overlay above a dimmed parent background with a fixed modal size and no footer. 2. **Content Auto-Resizing** • **Given** a parent screen with a configured “Open Screen in Pop Up” action • **When** I click the element that triggers “Open Screen in Pop Up” • **Then** the content within the screen first attempts to auto-scale or reflow to fit within the modal • **And** if any elements remain larger than the viewport vertical scrollbars appear so that all content remains accessible.

FR-201	As a user, I want the parent screen to remain rendered (dimmed) but inactive while the modal is open, so that I can focus on the modal's primary call to action without distractions.	AC 1. **Dimmed Background** • **Given** the modal overlay is open • **When** I view the screen • **Then** the parent screen's UI appears dimmed (50% opacity) behind the modal.
FR-203	As a user, I want clicking outside the modal overlay to have no effect, so that I don't accidentally dismiss the modal and lose my input.	AC 1. **Block Interaction Behind Modal** • **Given** the modal overlay is open • **When** I attempt to click, tap, or interact with any element on the parent screen • **Then** no action occurs and the parent screen remains inactive.
FR-204	As a user, I want the close button inside the modal to invoke a "Close Pop Up" action automatically when clicked, so that I return to the parent screen in a single step.	AC 1. **Successful Save Closes Modal** • **Given** I have entered valid data in the modal's form fields • **When** I click the close button • **Then** the data is persisted, the modal closes immediately,

		and focus returns to the parent screen **2. Validation Errors Keep Modal Open** • **Given** I have left required fields empty or entered invalid data in the modal • **When** I click the close button • **Then** field-level validation errors appear inside the modal, the modal remains open, and no “Close Pop Up” action is fired. 3. **Parent Data Preservation** • **Given** I have entered or modified data on the parent screen • **When** I open the modal and then close it—whether by clicking the close button or the “X” icon • **Then** all data I entered on the parent screen remains populated exactly as I left it (no fields are reset or lost).
FR-205	As a user, I want to click the “X” icon in the top-right corner of the modal to close it, so that I can explicitly dismiss the overlay and return to the parent screen.	AC 1. **Visibility of “X” Icon**

- **Given** the modal is open
- **When** I view the modal header
- **Then** I see a clearly identifiable “X” icon in the top-right corner.

2. **Close on Click**

- **Given** the modal is open with no fields filled in
- **When** I click the “X” icon
- **Then** the modal closes immediately, and focus returns to the parent screen

3. **Confirmation on Unsaved Changes**

- **Given** I have filled in one or more fields in the open modal without saving
- **When** I click the “X” icon in the top-right corner
- **Then** a confirmation dialog appears with the message “Are you sure you want to close without saving?” and, upon clicking “Yes,” the modal closes, all unsaved changes are discarded, and the

		parent screen remains unchanged. 4. **No Data Return on X Close** • **Given** the modal is open and the user clicks the “X” icon • **When** the modal closes • **Then** no modal-specific data or return payload is passed back to the parent screen
FR-206	As a user, I want to save data inside the modal and see the parent screen update accordingly, so that I know my changes persisted and reflected in context.	AC 1. **Successful Save Persists and Updates Parent** • **Given** I have entered valid data in the modal’s form fields • **When** I click the close button • **Then** the data is saved, the modal closes, and the parent screen’s affected component (e.g., grid) refreshes to show the updated record. **2. Validation Errors Keep Modal Open** • **Given** I have left required fields empty

		or entered invalid data in the modal • **When** I click the close button • **Then** field-level validation errors appear inside the modal, the modal remains open, and no changes are made to the parent screen.
FR-207	As a user, I want one modal to open at a time, so that I avoid nested overlays and can focus on completing the current task.	AC 1. **Block Second Modal Open** • **Given** a modal is already open • **When** I trigger another “Open Screen in Pop Up” action • **Then** no new modal opens and a show message appears with “Cannot open another modal while one is active”. 2. **No Visual Change** • **Given** I attempt to open a second modal while one is active • **When** that action is blocked • **Then** there is no visual change (no additional backdrop, dimming, or flicker).

FR-208 NOT DOING	As a user interacting with a modal, I want the modal to display only a single “Save and Close” button (removing all other navigation controls), so that I have one clear CTA and no confusing back or navigation buttons.	AC 1. **Single Action Button** • **Given** a screen is tied to a “Open Screen in Pop Up” action • **When** the modal renders in runtime • **Then** only the “Save and Close” button appears in the footer, and any original “Back,” “Save & Back,” or other navigation buttons are removed. 2. **Clean Header/Footer** • **Given** the modal is open • **When** I inspect the header and footer areas • **Then** no back-arrow, breadcrumb, or tertiary action controls are visible—only the “Save and Close” button in the footer and the “X” icon in the header.
FR-209	As a user, I want the “Open Screen As a Pop Up” action to pause the action sequence and, upon modal close, resume processing with returned results, so that I can branch logic based on user input collected in the modal.	AC 1. **Pause Action Sequence** • **Given** an action set where “Open Screen in Pop Up” is followed by other actions (e.g., “Update Record”)

		• **When** the “Open Screen in Pop Up” action runs at runtime • **Then** subsequent actions do not execute until the modal is closed. 2. **Resume Action Sequence** • **Given** an action set where “Open Screen in Pop Up” is followed by other actions (e.g., “Update Record”) • **When** the “Close Pop Up” action runs at runtime • **Then** the remaining actions in the action set executes immediately.
FR-209	As a user, I want refreshing the browser while a modal is open to reload the page back to the parent screen, just like show message does today.	AC 1. **Full-Page Reload Behaviour** ◦ **Given** a modal overlay is open in the browser ◦ **When** I click the browser’s Refresh/Reload button ◦ **Then** the entire page reloads and displays the parent screen (no modal

		overlay remains just like show message). 2. **State Restoration** ◦ **Given** I had unsaved changes in the modal before refreshing ◦ **When** the page reloads ◦ **Then** those changes are discarded, and the parent screen loads its default or last-saved state.

3. **Mobile Devices**

ID	User Story	Acceptance Criteria
FR-300 DEFERRED	As a user on a mobile, I want the modal to resize to fit my screen, so that I can see all content with vertical scrolling if required.	AC
FR-301 DEFERRED	As a user on mobile, I want the modal's body to scroll independently of the page behind it, so that I can access content without unintentionally scrolling the parent page.	AC 1. **Independent Body Scrolling** • **Given** the modal is open on a mobile device • **When** I swipe up or down within the modal content area • **Then** only the modal's body content scrolls, and the parent page remains fixed. **2. No Background Scroll**

		• **Given** the modal is open and content exceeds viewport height • **When** I continue swiping after reaching the top or bottom of the modal's content • **Then** the parent page does not scroll (i.e., the swipe is blocked at the modal's boundary).
FR-302 DEFERRED	As a user on mobile, I want the modal to adjust when the on-screen keyboard appears (input fields remain visible), so that I can type without fields being hidden behind the keyboard.	AC 1. **Input Field Remains Visible** • **Given** the modal is open on a mobile device and I tap into a text input • **When** the on-screen keyboard appears • **Then** the modal container shifts or resizes so that the focused input remains fully visible above the keyboard. 2. **Modal Content Scroll to Focus** • **Given** the modal contains inputs positioned near the bottom of the viewport • **When** I focus a lower input and the keyboard

		opens • **Then** the modal's body automatically scrolls so the input sits at least X px above the keyboard. **3. No Unintended Modal Dismissal** • **Given** the keyboard is open and I tap outside the focused input but still within the modal (e.g., to dismiss the keyboard) • **When** that tap is registered • **Then** the keyboard hides and the modal remains open, without automatically closing.
FR-303 DEFERRED	As a user on mobile, I want taps and swipes inside the modal to behave consistently (no accidental closures), so that I can interact reliably.	AC 1. **No Accidental Close on Swipe** • **Given** the modal is open on a mobile device • **When** I swipe up or down within the modal content area • **Then** the modal does not close; instead, the content scrolls or the swipe is ignored. **2. Single-Tap Activation** • **Given** I tap a "Save" or "X" button inside the

		modal • **When** I perform a single tap • **Then** the associated action fires exactly once (debounced) and the modal responds appropriately. **3. Background Tap Ignored** • **Given** the modal is open and a backdrop surrounds it • **When** I tap on the backdrop (outside the modal container) • **Then** no action occurs, and the modal remains open (per desktop behaviour).
FR-304	As a user on mobile, I want a modal-configured screen to display as a full-screen overlay (not a separate screen navigation), so that I can interact with it natively on my device without cramped overlays.	AC 1. **Full-Screen Modal Overlay** • **Given** I am on a mobile device • **When** I tap an element with “Open Screen in Pop Up” • **Then** the modal screen appears as a full-screen overlay, not as a new navigation. 2. **Modal Close Controls**

		• **Given** the full-screen modal is open • **When** I tap the “X” icon or the close button (same functionality on desktop) • **Then** the modal overlay closes, and focus returns to the parent screen

4. **Access Control**

ID	User Story	Acceptance Criteria
FR-400	As a user, I want the system to check my access before opening a modal, so that screens I’m authorized for open in a popup, and attempts on unauthorized screens redirect to “Get Access” screen.	AC 1. **Open Modal When Authorized** • **Given** I attempt to open a modal from a parent screen • **When** the system checks my permissions and confirms I’m authorized • **Then** the modal overlay opens the specified screen centered over a dimmed parent background. 2. **Deny Modal When Unauthorized**

		• **Given** I attempt to open a modal from a parent screen • **When** the system checks my permissions and finds I'm unauthorized • **Then** the "Get Access" screen appears informing the user to contact their account administrator for permission.

5. **Component**

Component	Acceptance Criteria
Input Fields	Should be able to account for compulsory and optional fields
Buttons	Primary, secondary and tertiary buttons
Header Copy	Standard sizes for H1, H2, H3
Body Copy	Standard sizes for H4, H5, H6
Footer	To house buttons
Closed Icons	To be able to click out of the modal
Icons	Should be able to select from icon pack
Toggle	N/A
Check Box	N/A
Table	TBD
Card	TBD

Non-Functional Requirements

The following non-functional requirements outline performance, responsiveness, accessibility, and security criteria for the modal component.

Category	Requirement
Performance	Modal open time ≤ 100ms on typical data payloads
Responsiveness	Must function on desktop/tablet/mobile
Accessibility	Color contrast and focus states must comply with WCAG 2.1 AA standards
Security	Data visibility must respect role-based access controls and user permissions

---

Release Plan & Timeline

Approach

We will release the *Screen-as-Modal* capability as a **general availability (GA)** feature to all eligible accounts from day one. A broad GA launch makes sense because this feature is purely additive and is low-risk of breaking existing workflows. Launching broadly will maximize adoption signals, accelerate feedback, and validate user demand more quickly. It also encourages diverse usage patterns across our customer base, helping to ensure that improvements are grounded in real-world workflows.

Additionally, ~ 250 unique users actively use the screen designer (customizer) each week — which provides a robust sample size for gathering meaningful feedback. Since the majority of valuable and successful customizations are delivered by partners or the Professional Services team, there is minimal risk that general users will misuse or break critical workflows. In addition, usage will naturally surface any bugs, which is a positive signal that people are engaging with the feature.

Timeline

Phase	Approach & Tactics	Dates
Pre-GA Enablement & Training	Deliver short demo + walkthrough on how to	TBD

	configure modals to Partners/PS team to prepare them for building with modals at GA	
General Availability Release	Release to all Method accounts (Customers, Partners, & PS) with clear onboarding and enablement for builders	TBD
Rollback Plan (if needed)	Disable “Open Screen in Pop Up” globally while preserving screens, with rapid comms to PS/partners	As needed
Post-Launch Monitoring & Metrics Tracking	Track modal adoption and usage success metrics, monitor support tickets, and Partner/PS team feedback	Ongoing
Retrospective & Iteration Planning	Analyze adoption signals, usage patterns, support/PS/Partner feedback and prioritize next enhancements	Ongoing

Stakeholder Communication

Internal (PS, Support, Sales, Partners)

- Link to a dedicated Help Center article with a short video walkthrough
- Deliver short demo + walkthrough on how to configure modals

Customer Communication

- Release note in the next product newsletter
- Link to a dedicated Help Center article with a short video walkthrough

Rollback Strategy

Approach:
While Screen-as-a-Modal is additive and low-risk, a contingency rollback plan is in place in case

of:

- critical performance regressions
- major customer-impacting bugs
- unexpected compatibility issues with heavily customized environments

Rollback Steps:
✅ Disable the *"Open Screen in Pop Up"* action in the action editor
✅ Preserve all user-created screens and data
✅ Communicate rollback status and expected next steps within 1 business day
✅ Coordinate re-enablement plan with Engineering and Support

Out of Scope

The following items **will not** be addressed in this sprint to keep the modal focused and deliverable:

- Navigation between screen to screen within a modal
- Modal within a modal
- Tab inside a modal
- Modal inside a tab
- Signature Widget

Pressing Questions

- By setting a screen as a modal, does it also lock down certain functionality?
- What happens if a user doesn't include a Save (or Update) button on a modal screen? How should the modal close?
- When a Show Modal action is in a flow alongside other actions (e.g., Send Email), does the modal block the rest of the actions until it's closed, or do they fire immediately?
- After closing a modal, how does the parent screen or its controls refresh? For complex controls (grids, charts, galleries, calendars), there's built-in auto-refresh, but what about simpler screens?
- When showing a screen as a modal, how do we hide or reposition certain elements (e.g., title bar, Save/Save-And-Close, Back button)?
- How does a user know that they can't navigate within a modal (modal to modal)? What if they want to create a wizard?

Open Questions

- If partners and customers build hundreds of modals as screens, how can we ever deprecate this approach without breaking live customizations?
- Without a clear migration path to a true “Modal widget” in the future App Builder, how do we prevent this workaround from becoming a permanent, sub-optimal solution?
- Without a clear notification message or any indication that the modal closed successfully, how can users know if their work saved?
- How should we handle device rotation mid-modal (e.g., from portrait → landscape)? Does the modal reflow correctly, and do we retain scroll position?
- How do we curate the list of “eligible” screens for success metrics?